

Recorridos de Árbol Binario en Java

¿Qué es un recorrido en un árbol binario?

Un recorrido es el proceso de visitar sistemáticamente todos los nodos de un árbol binario, generalmente para realizar alguna acción con sus valores (como imprimirlos, ordenarlos o procesarlos). Existen varios tipos de recorridos, que se diferencian por el orden en que se visitan los nodos: raíz, subárbol izquierdo y subárbol derecho.

Definición de la clase Nodo

Antes de explicar los recorridos, se define la estructura básica del árbol binario en Java:

```
1 // Clase Nodo: representa un nodo del árbol binario
2 class Nodo {
3     int valor;                // El dato que almacena el nodo
4     Nodo izquierdo, derecho;  // Referencias al hijo izquierdo y
5                               // derecho
6
7     public Nodo(int valor) {
8         this.valor = valor;
9         izquierdo = derecho = null;
10    }
11 }
```

Recorrido Inorden (In-order)

Definición y utilidad

- Orden de visita: **Izquierda** → **Raíz** → **Derecha**. - Se usa frecuentemente en árboles binarios de búsqueda (ABB), ya que devuelve los datos en orden ascendente.

Código Java comentado

```

1 // Recorrido Inorden
2 void inorden(Nodo nodo) {
3     if (nodo != null) {
4         inorden(nodo.izquierdo);    // Paso 1: Recorrer sub rbol
4         izquierdo
5         System.out.print(nodo.valor + " "); // Paso 2: Visitar la
5         ra z
6         inorden(nodo.derecho);      // Paso 3: Recorrer sub rbol
6         derecho
7     }
8 }

```

Ejemplo de uso: Imprimir todos los valores del árbol en orden creciente.

Recorrido Preorden (Pre-order)

Definición y utilidad

- Orden de visita: **Raíz** → **Izquierda** → **Derecha**. - Útil para clonar o copiar un árbol, generar expresiones prefix o guardar su estructura.

Código Java comentado

```

1 // Recorrido Preorden
2 void preorden(Nodo nodo) {
3     if (nodo != null) {
4         System.out.print(nodo.valor + " "); // Paso 1: Visitar la
4         ra z
5         preorden(nodo.izquierdo);           // Paso 2: Recorrer
5         sub rbol izquierdo
6         preorden(nodo.derecho);             // Paso 3: Recorrer
6         sub rbol derecho
7     }
8 }

```

Ejemplo de uso: Generar un archivo con la estructura del árbol.

Recorrido Postorden (Post-order)

Definición y utilidad

- Orden de visita: **Izquierda** → **Derecha** → **Raíz**. - Se usa para liberar memoria, evaluar expresiones en postfijo o eliminar el árbol desde las hojas.

Código Java comentado

```
1 // Recorrido Postorden
2 void postorden(Nodo nodo) {
3     if (nodo != null) {
4         postorden(nodo.izquierdo);    // Paso 1: Recorrer sub rbol
5         postorden(nodo.derecho);      // Paso 2: Recorrer sub rbol
6         System.out.print(nodo.valor + " "); // Paso 3: Visitar la
7     }
8 }
```

Ejemplo de uso: Eliminar el árbol correctamente desde las hojas hacia la raíz.

Recorrido por Niveles (Breadth-First / BFS)

Definición y utilidad

- Orden de visita: **Nivel por nivel**, de izquierda a derecha. - Se utiliza una estructura de tipo cola (Queue) para mantener el orden. - Útil para determinar la profundidad de un árbol o imprimirlo por capas.

Código Java comentado

```
1 import java.util.LinkedList;
2 import java.util.Queue;
3
4 // Recorrido por niveles (BFS)
5 void porNiveles(Nodo raiz) {
6     if (raiz == null) return;
7
8     Queue<Nodo> cola = new LinkedList<>();
9     cola.add(raiz); // Paso 1: Agregar la raíz
10
11     while (!cola.isEmpty()) {
12         Nodo actual = cola.poll(); // Paso 2: Extraer el primer
13         System.out.print(actual.valor + " ");
14
15         if (actual.izquierdo != null)
16             cola.add(actual.izquierdo); // Paso 3: Encolar hijo
17         if (actual.derecho != null)
```

```

18         cola.add(actual.derecho);    // Paso 4: Encolar hijo
19         derecho
20     }
}

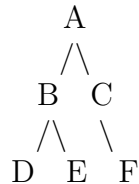
```

Ejemplo de uso: Visualizar el árbol como una lista de niveles (útil en impresión gráfica).

Resumen comparativo de los recorridos

- **Inorden:** Devuelve los elementos ordenados (si es ABB).
 - **Preorden:** Útil para copiar o representar árboles.
 - **Postorden:** Se usa para eliminar o evaluar árboles.
 - **Por niveles:** Muestra la estructura por profundidad.
-

Ejemplo visual del árbol



Resultados esperados de los recorridos:

- Inorden: D B E A C F
 - Preorden: A B D E C F
 - Postorden: D E B F C A
 - Por niveles: A B C D E F
-

Conclusión

Los recorridos de árboles binarios son fundamentales para manipular estructuras jerárquicas en la programación. Entender el orden de visita de los nodos permite aplicar técnicas adecuadas para búsqueda, ordenamiento, evaluación o visualización de estructuras.